

[VN] VPC là gì?

[EN] What is a VPC?

[VN] VPC (Virtual Private Cloud) là một mạng ảo riêng trong AWS, nơi bạn có thể triển khai tài nguyên như EC2, RDS hoặc Lambda trong một môi trường mạng được kiểm soát. Trong VPC, bạn có thể cấu hình subnet (public và private), route table, internet gateway và security group để kiểm soát traffic vào và ra. VPC giúp cô lập hệ thống, tăng bảo mật và cho phép thiết kế kiến trúc mạng linh hoạt giống như một data center riêng trên cloud. Nhờ đó, bạn có thể kiểm soát IP range, phân vùng mạng và chính sách truy cập theo nhu cầu hệ thống.

[EN] VPC (Virtual Private Cloud) is a private virtual network in AWS where you can deploy resources such as EC2, RDS, or Lambda in a controlled networking environment. Inside a VPC, you can configure subnets (public and private), route tables, internet gateways, and security groups to manage inbound and outbound traffic. VPC provides isolation, improved security, and flexible network architecture similar to a private data center in the cloud. This allows you to control IP ranges, network segmentation, and access policies based on system requirements.

[VN] Horizontal scaling vs vertical scaling

[EN] Horizontal scaling vs vertical scaling

[VN] Horizontal scaling là cách tăng khả năng chịu tải bằng cách thêm nhiều instance hoặc nhiều server vào hệ thống, ví dụ tăng từ 1 lên 10 service instance phía sau load balancer. Cách này giúp hệ thống chịu lỗi tốt hơn và dễ mở rộng trong môi trường cloud, nhưng cần thiết kế stateless và xử lý distributed system phức tạp hơn. Vertical scaling là tăng tài nguyên cho một server duy nhất như thêm CPU, RAM hoặc ổ đĩa. Cách này đơn giản hơn và không cần thay đổi nhiều về kiến trúc, nhưng có giới hạn phần cứng và rủi ro single point of failure. Vì vậy, hệ thống hiện đại thường ưu tiên horizontal scaling để đạt tính linh hoạt và high availability.

[EN] Horizontal scaling means increasing system capacity by adding more instances or servers, for example scaling from 1 to 10 service instances behind a load balancer. This approach improves fault tolerance and works well in cloud environments, but it requires stateless design and handling distributed system complexity. Vertical scaling means upgrading a single server by adding more CPU, RAM, or storage. It is simpler and does not require major architecture changes, but it has hardware limits and creates a single point of failure. Therefore, modern systems usually prefer horizontal scaling for better flexibility and high availability.

[VN] EC2 vs Lambda.

[EN] EC2 vs Lambda.

[VN] EC2 là dịch vụ máy ảo, nơi bạn tự quản lý server, hệ điều hành, scaling và cấu hình ứng dụng. Nó phù hợp cho hệ thống chạy lâu dài, cần kiểm soát cao hoặc workload phức tạp. Lambda là dịch vụ serverless, chỉ chạy code khi có event và tự động scale theo request, bạn không cần quản lý server. Lambda phù hợp cho tác vụ ngắn, event-driven hoặc xử lý theo request không liên tục. EC2 linh hoạt và kiểm soát tốt hơn, nhưng cần quản lý nhiều hơn; Lambda đơn giản, tự động scale nhưng có giới hạn về thời gian chạy và tài nguyên.

[EN] EC2 is a virtual machine service where you manage the server, operating system, scaling, and application configuration yourself. It is suitable for long-running systems that require high control or complex workloads. Lambda is a serverless service that runs code only when triggered by events and automatically scales based on requests, without server management. Lambda is ideal for short tasks, event-driven workloads, or intermittent requests. EC2 offers more flexibility and control but requires more management, while Lambda is simpler and auto-scaling but has limits on execution time and resources.

[VN] SQS vs SNS.

[EN] SQS vs SNS.

[VN] SQS là dịch vụ message queue dùng để lưu trữ message và cho phép consumer xử lý theo cơ chế pull. Message được giữ trong queue cho đến khi được xử lý và xóa, phù hợp cho xử lý bất đồng bộ và đảm bảo độ tin cậy. SNS là dịch vụ pub/sub dùng để gửi message đến nhiều subscriber cùng lúc theo cơ chế push, ví dụ gửi đến SQS, Lambda hoặc HTTP endpoint. SQS phù hợp cho xử lý tuần tự và kiểm soát retry, còn SNS phù hợp cho broadcast sự kiện đến nhiều hệ thống khác nhau. Trong thực tế, SNS và SQS thường được kết hợp với nhau để xây dựng kiến trúc event-driven.

[EN] SQS is a message queue service that stores messages and allows consumers to process them using a pull model. Messages remain in the queue until they are processed and deleted, making it suitable for asynchronous processing and reliability. SNS is a pub/sub service that pushes messages to multiple subscribers at the same time, such as SQS, Lambda, or HTTP endpoints. SQS is suitable for sequential processing and retry control, while SNS is ideal for broadcasting events to multiple systems. In practice, SNS and SQS are often combined to build event-driven architectures.

[VN] Load balancing L4 vs L7 khác nhau thế nào?

[EN] How does L4 vs L7 load balancing differ?

[VN] Load balancing L4 hoạt động ở tầng Transport (TCP/UDP), nó phân phối traffic dựa trên IP và port mà không hiểu nội dung bên trong request. Vì vậy L4 nhanh và hiệu năng

cao, phù hợp cho hệ thống cần throughput lớn và xử lý đơn giản. Load balancing L7 hoạt động ở tầng Application (HTTP/HTTPS), nó hiểu nội dung request như URL, header, cookie và có thể routing theo path hoặc domain. L7 linh hoạt hơn, hỗ trợ SSL termination, authentication và routing thông minh, nhưng có overhead cao hơn L4. Tóm lại, L4 tối ưu cho hiệu năng và đơn giản, còn L7 tối ưu cho routing phức tạp và logic ở tầng application.

[EN] L4 load balancing works at the Transport layer (TCP/UDP) and distributes traffic based on IP address and port without understanding the request content. Because of this, L4 is fast and efficient, suitable for systems that require high throughput and simple routing. L7 load balancing works at the Application layer (HTTP/HTTPS) and understands request details such as URL, headers, and cookies, allowing routing based on path or domain. L7 is more flexible and supports features like SSL termination, authentication, and intelligent routing, but it has more overhead than L4. In summary, L4 is optimized for performance and simplicity, while L7 is better for complex routing and application-level logic.

[VN] Auto Scaling Group hoạt động ra sao?

[EN] How does an Auto Scaling Group work?

[VN] Auto Scaling Group (ASG) trong AWS là dịch vụ tự động quản lý số lượng EC2 instance theo cấu hình mong muốn. ASG có thể scale out (tăng instance) khi CPU cao hoặc traffic tăng, và scale in (giảm instance) khi tải thấp dựa trên policy và metric như CPU utilization hoặc request count. Nó cũng đảm bảo luôn duy trì số lượng instance tối thiểu; nếu một instance bị lỗi, ASG sẽ tự động tạo instance mới thay thế. ASG thường kết hợp với Load Balancer để phân phối traffic đều giữa các instance, giúp hệ thống ổn định và linh hoạt theo tải thực tế.

[EN] Auto Scaling Group (ASG) in AWS is a service that automatically manages the number of EC2 instances based on a desired configuration. ASG can scale out (add instances) when CPU usage or traffic increases, and scale in (remove instances) when the load decreases, based on policies and metrics such as CPU utilization or request count. It also ensures a minimum number of instances is always running; if an instance fails, ASG automatically launches a new one to replace it. ASG is commonly used with a Load Balancer to distribute traffic evenly across instances, providing stability and flexibility according to real workload.

[VN] Blue-green vs Canary deployment khác nhau thế nào?

[EN] How are Blue-green and Canary deployments different?

[VN] Blue-green deployment là chiến lược triển khai với hai môi trường giống nhau: “blue” (version hiện tại) và “green” (version mới). Khi version mới đã sẵn sàng và được

kiểm tra, toàn bộ traffic sẽ được chuyển sang môi trường mới gần như ngay lập tức. Cách này giúp rollback rất nhanh bằng cách chuyển traffic ngược lại, nhưng cần nhiều tài nguyên vì phải duy trì hai môi trường hoàn chỉnh. Canary deployment là chiến lược phát hành version mới cho một phần nhỏ người dùng trước (ví dụ 5–10%), sau đó tăng dần nếu hệ thống hoạt động ổn định. Cách này giúp phát hiện lỗi sớm trong production và giảm rủi ro, nhưng việc triển khai và quản lý traffic phức tạp hơn.

[EN] Blue-green deployment uses two identical environments: “blue” (current version) and “green” (new version). When the new version is ready and tested, all traffic is switched to the new environment almost instantly. This allows very fast rollback by switching traffic back, but it requires more resources because two full environments must run at the same time. Canary deployment releases the new version to a small percentage of users first (for example 5–10%), and then gradually increases the traffic if everything works well. This approach helps detect problems early in production and reduces risk, but traffic control and deployment are more complex.

[VN] Cách quản lý configuration theo môi trường (dev/staging/prod)?

[EN] How to manage configuration by environment (dev/staging/prod)?

[VN] Để quản lý configuration theo môi trường như dev, staging, prod, bạn nên tách config ra khỏi code và dùng biến môi trường (environment variables) để lưu các giá trị như database URL, secret key hoặc API endpoint. Có thể dùng file .env cho môi trường local và cấu hình trực tiếp trên server hoặc container trong production. Ngoài ra, nên có file cấu hình riêng cho từng môi trường hoặc dùng một config loader để đọc biến môi trường và map vào object cấu hình. Quan trọng là không hard-code thông tin nhạy cảm trong source code và đảm bảo mỗi môi trường có cấu hình độc lập.

[EN] To manage configuration across environments such as dev, staging, and prod, you should separate configuration from the code and use environment variables to store values like database URL, secret keys, or API endpoints. You can use a .env file for local development and configure variables directly on the server or container in production. It is also common to have separate config files per environment or use a config loader that reads environment variables and maps them into a configuration object. The key point is to avoid hard-coding sensitive information in the source code and ensure each environment has its own independent configuration.

[VN] Làm sao thiết kế system HA trên AWS?

[EN] How to design a HA system on AWS?

[VN] Để thiết kế hệ thống High Availability (HA) trên AWS, cần triển khai tài nguyên trên nhiều Availability Zone để tránh single point of failure. Tôi sẽ đặt application server sau

Load Balancer và cấu hình Auto Scaling Group để tự động tăng hoặc thay thế instance khi có lỗi. Database có thể dùng RDS Multi-AZ hoặc DynamoDB với replication tự động. Ngoài ra, cần cấu hình health check, backup định kỳ và monitoring để phát hiện sự cố sớm. Thiết kế stateless service và lưu session hoặc state trong Redis hoặc database cũng giúp hệ thống dễ scale và chịu lỗi tốt hơn.

[EN] To design a High Availability (HA) system on AWS, resources should be deployed across multiple Availability Zones to avoid a single point of failure. I would place application servers behind a Load Balancer and configure an Auto Scaling Group to automatically scale or replace failed instances. For the database, I can use RDS Multi-AZ or DynamoDB with built-in replication. It is also important to configure health checks, regular backups, and monitoring to detect issues early. Designing stateless services and storing session or state in Redis or a database also improves scalability and fault tolerance.

[VN] Làm sao thiết kế hệ thống chịu được AZ failure?

[EN] How to design a system resilient to AZ failure?

[VN] Để thiết kế hệ thống chịu được AZ failure trên AWS, cần triển khai tài nguyên trên nhiều Availability Zone thay vì chỉ một AZ. Application server nên chạy trong Auto Scaling Group và đặt sau Load Balancer để tự động chuyển traffic sang AZ còn hoạt động khi một AZ bị lỗi. Database có thể cấu hình RDS Multi-AZ hoặc sử dụng DynamoDB với replication tự động. Ngoài ra, nên lưu trữ file trên S3 và thiết kế service theo hướng stateless để dễ dàng scale và failover. Monitoring và health check cũng rất quan trọng để phát hiện và chuyển đổi nhanh khi có sự cố.

[EN] To design a system that can handle an AZ failure on AWS, resources should be deployed across multiple Availability Zones instead of just one. Application servers should run in an Auto Scaling Group behind a Load Balancer to automatically redirect traffic to healthy AZs if one fails. The database can use RDS Multi-AZ or DynamoDB with built-in replication. Files should be stored in S3, and services should be designed as stateless to support easy scaling and failover. Monitoring and health checks are also important to detect failures quickly and trigger automatic recovery.

[VN] Nếu migrate hệ thống on-premise lên AWS, bạn sẽ làm gì đầu tiên?

[EN] If migrating an on-premise system to AWS, what would you do first?

[VN] Nếu migrate hệ thống on-premise lên AWS, việc đầu tiên tôi làm là đánh giá toàn bộ hệ thống hiện tại: kiến trúc, dependency, database, network, bảo mật và hiệu năng. Sau đó, tôi xác định chiến lược migrate phù hợp như rehost (lift and shift), replatform hoặc refactor. Tiếp theo, tôi thiết kế kiến trúc target trên AWS bao gồm VPC, security, IAM và backup. Trước khi migrate chính thức, tôi sẽ test trên môi trường staging, kiểm tra hiệu

năng và kế hoạch rollback để giảm rủi ro. Bước chuẩn bị kỹ càng giúp quá trình migrate an toàn và ít gián đoạn.

[EN] If migrating an on-premise system to AWS, the first step I would take is to assess the current system, including architecture, dependencies, databases, network, security, and performance. Then, I would choose a suitable migration strategy such as rehost (lift and shift), replatform, or refactor. Next, I would design the target architecture on AWS, including VPC, security, IAM, and backup. Before the actual migration, I would test in a staging environment, validate performance, and prepare a rollback plan to reduce risk. Careful preparation ensures a safe migration with minimal disruption.

VII. Cloud & Infrastructure

VPC là gì?

Horizontal scaling vs vertical scaling.

EC2 vs Lambda.

SQS vs SNS.

Load balancing L4 vs L7 khác nhau thế nào?

Auto Scaling Group hoạt động ra sao?

Blue-green vs Canary deployment khác nhau thế nào?

Cách quản lý configuration theo môi trường (dev/staging/prod)?

Làm sao thiết kế system HA trên AWS?

Làm sao thiết kế hệ thống chịu được AZ failure?

Nếu migrate hệ thống on-premise lên AWS, bạn sẽ làm gì đầu tiên?